JavaScript is disabled on your browser.

Java SE 27 & JDK 27

Class StackTraceElement

java.lang.Object
    java.lang.StackTraceElement

All Implemented Interfaces:

Serializable

```
public final class StackTraceElement
extends Object
implements Serializable
```

An element in a stack trace, as returned by `Throwable.getStackTrace()`. Each element represents a single stack frame. All stack frames except for the one at the top of the stack represent a method invocation. The frame at the top of the stack represents the execution point at which the stack trace was generated. Typically, this is the point at which the throwable corresponding to the stack trace was created.

Since:

1.4

See Also:

Serialized Form

Constructor Summary

Constructors

Constructor	Description
StackTraceElement (String declaringClass, String methodName, String fileName, int lineNumber)	Creates a stack trace element representing the specified execution point.
StackTraceElement (String classLoaderName, String moduleName, String moduleVersion, String declaringClass, String methodName, String fileName, int lineNumber)	Creates a stack trace element representing the specified execution point.

Method Summary

All Methods | **Instance Methods**

Modifier and Type	Method	Description
boolean	**equals**(Object obj)	Returns true if the specified object is another StackTraceElement instance representing the same execution point as this instance.
String	**getClassLoaderName**()	Returns the name of the class loader of the class containing the execution point represented by this stack trace element.
String	**getClassName**()	Returns the binary name of the Class containing the execution point represented by this stack trace element.
String	**getFileName**()	Returns the name of the source file containing the execution point represented by this stack trace element.
int	**getLineNumber**()	Returns the line number of the source line containing the execution point represented by this stack trace element.
String	**getMethodName**()	Returns the name of the method containing the execution point represented by this stack trace element.
String	**getModuleName**()	Returns the module name of the module containing the execution point represented by this stack trace element.

String	**getModuleVersion**()	Returns the module version of the module containing the execution point represented by this stack trace element.
int	**hashCode**()	Returns a hash code value for this stack trace element.
boolean	**isNativeMethod**()	Returns true if the method containing the execution point represented by this stack trace element is a native method.
String	**toString**()	Returns a string representation of this stack trace element.

Methods declared in class Object

clone, finalize, getClass, notify, notifyAll, wait, wait, wait

Constructor Details

StackTraceElement

```
public StackTraceElement(String declaringClass,
                         String methodName,
                         String fileName,
                         int lineNumber)
```

Creates a stack trace element representing the specified execution point. The module name and module version of the stack trace element will be null.

Parameters:

declaringClass - the binary name of the class containing the execution point represented by the stack trace element

methodName - the name of the method containing the execution point represented by the stack trace element

fileName - the name of the file containing the execution point represented by the stack trace element, or null if this information is unavailable

`lineNumber` - the line number of the source line containing the execution point represented by this stack trace element, or a negative number if this information is unavailable. A value of -2 indicates that the method containing the execution point is a native method

Throws:

`NullPointerException` - if `declaringClass` or `methodName` is null

Since:

1.5

StackTraceElement

```
public StackTraceElement(String classLoaderName,
                         String moduleName,
                         String moduleVersion,
                         String declaringClass,
                         String methodName,
                         String fileName,
                         int lineNumber)
```

Creates a stack trace element representing the specified execution point.

Parameters:

`classLoaderName` - the class loader name if the class loader of the class containing the execution point represented by the stack trace is named; otherwise `null`

`moduleName` - the module name if the class containing the execution point represented by the stack trace is in a named module; otherwise `null`

`moduleVersion` - the module version if the class containing the execution point represented by the stack trace is in a named module that has a version; otherwise `null`

`declaringClass` - the binary name of the class containing the execution point represented by the stack trace element

`methodName` - the name of the method containing the execution point represented by the stack trace element

`fileName` - the name of the file containing the execution point represented by the stack trace element, or `null` if this information is unavailable

`lineNumber` - the line number of the source line containing the execution point represented by this stack trace element, or a negative number if this information is unavailable. A value of -2 indicates that the method containing the execution point is a native method

Throws:

`NullPointerException` - if `declaringClass` is `null` or `methodName` is `null`

Since:

9

Method Details

getFileName

```
public String getFileName()
```

Returns the name of the source file containing the execution point represented by this stack trace element. Generally, this corresponds to the `SourceFile` attribute of the relevant `class` file (as per *The Java Virtual Machine Specification*, Section §4.7.7). In some systems, the name may refer to some source code unit other than a file, such as an entry in source repository.

Returns:

the name of the file containing the execution point represented by this stack trace element, or `null` if this information is unavailable.

getLineNumber

```
public int getLineNumber()
```

Returns the line number of the source line containing the execution point represented by this stack trace element. Generally, this is derived from the `LineNumberTable` attribute of the relevant `class` file (as per *The Java Virtual Machine Specification*, Section §4.7.8).

Returns:

the line number of the source line containing the execution point represented by this stack trace element, or a negative number if this information is unavailable.

getModuleName

```
public String getModuleName()
```

Returns the module name of the module containing the execution point represented by this stack trace element.

Returns:

the module name of the `Module` containing the execution point represented by this stack trace element; `null` if the module name is not available.

Since:

9

See Also:

`Module.getName()`

getModuleVersion

```
public String getModuleVersion()
```

Returns the module version of the module containing the execution point represented by this stack trace element.

Returns:

the module version of the `Module` containing the execution point represented by this stack trace element; `null` if the module version is not available.

Since:

9

See Also:

`ModuleDescriptor.Version`

getClassLoaderName

```
public String getClassLoaderName()
```

Returns the name of the class loader of the class containing the execution point represented by this stack trace element.

Returns:

the name of the class loader of the class containing the execution point represented by this stack trace element; `null` if the class loader is not named.

Since:

9

See Also:

`ClassLoader.getName()`

getClassName

```
public String getClassName()
```

Returns the binary name of the `Class` containing the execution point represented by this stack trace element.

Returns:

the binary name of the `Class` containing the execution point represented by this stack trace element

getMethodName

```
public String getMethodName()
```

Returns the name of the method containing the execution point represented by this stack trace element. If the execution point is contained in an instance or class initializer, this method will return the appropriate *special method name*, "<init>" or "<clinit>", as per Section §3.9 of *The Java Virtual Machine Specification*.

Returns:

the name of the method containing the execution point represented by this stack trace element.

isNativeMethod

```
public boolean isNativeMethod()
```

Returns true if the method containing the execution point represented by this stack trace element is a native method.

Returns:

`true` if the method containing the execution point represented by this stack trace element is a native method.

toString

```
public String toString()
```

Returns a string representation of this stack trace element.

Overrides:

`toString` in class `Object`

API Note:

The format of this string depends on the implementation, but the following examples may be regarded as typical:

- `"com.foo.loader/foo@9.0/com.foo.Main.run(Main.java:101)"` - See the description below.
- `"com.foo.loader/foo@9.0/com.foo.Main.run(Main.java)"` - The line number is unavailable.
- `"com.foo.loader/foo@9.0/com.foo.Main.run(Unknown Source)"` - Neither the file name nor the line number is available.
- `"com.foo.loader/foo@9.0/com.foo.Main.run(Native Method)"` - The method containing the execution point is a native method.

- `"com.foo.loader//com.foo.bar.App.run(App.java:12)"` - The class of the execution point is defined in the unnamed module of the class loader named `com.foo.loader`.
- `"acme@2.1/org.acme.Lib.test(Lib.java:80)"` - The class of the execution point is defined in `acme` module loaded by a built-in class loader such as the application class loader.
- `"MyClass.mash(MyClass.java:9)"` - `MyClass` class is on the application class path.

The first example shows a stack trace element consisting of three elements, each separated by `"/"`, followed by the source file name and the line number of the source line containing the execution point. The first element `"com.foo.loader"` is the name of the class loader. The second element `"foo@9.0"` is the module name and version. The third element is the method containing the execution point; `"com.foo.Main""` is the binary name and `"run"` is the name of the method. `"Main.java"` is the source file name and `"101"` is the line number.

If a class is defined in an *unnamed module* then the second element is omitted as shown in `"com.foo.loader//com.foo.bar.App.run(App.java:12)"`.

If the class loader is a built-in class loader or is not named then the first element and its following `"/"` are omitted as shown in `"acme@2.1/org.acme.Lib.test(Lib.java:80)"`. If the first element is omitted and the module is an unnamed module, the second element and its following `"/"` are also omitted as shown in `"MyClass.mash(MyClass.java:9)"`.

The `toString` method may return two different values on two `StackTraceElement` instances that are equal, for example one created via the constructor, and one obtained from `Throwable` or `StackWalker.StackFrame`, where an implementation may choose to omit some element in the returned string.

Returns:

a string representation of the object

See Also:

`Throwable.printStackTrace()`

equals

```
public boolean equals(Object obj)
```

Returns true if the specified object is another `StackTraceElement` instance representing the same execution point as this instance. Two stack trace elements `a` and `b` are equal if and only if:

```
    equals(a.getClassLoaderName(), b.getClassLoaderName()) &&
    equals(a.getModuleName(), b.getModuleName()) &&
    equals(a.getModuleVersion(), b.getModuleVersion()) &&
    equals(a.getClassName(), b.getClassName()) &&
    equals(a.getMethodName(), b.getMethodName())
    equals(a.getFileName(), b.getFileName()) &&
    a.getLineNumber() == b.getLineNumber()
```

where `equals` has the semantics of `Objects.equals`.

Overrides:

`equals` in class `Object`

Parameters:

`obj` - the object to be compared with this stack trace element.

Returns:

true if the specified object is another `StackTraceElement` instance representing the same execution point as this instance.

See Also:

`Object.hashCode()`, `HashMap`

hashCode

```
public int hashCode()
```

Returns a hash code value for this stack trace element.

Overrides:

`hashCode` in class `Object`

Returns:

a hash code value for this object

See Also:

`Object.equals(java.lang.Object)`,
`System.identityHashCode(Object)`

Report a bug or suggest an enhancement
For further API reference and developer documentation see the Java SE Documentation, which contains more detailed, developer-targeted descriptions with conceptual overviews, definitions of terms, workarounds, and working code examples. Other versions.
Java is a trademark or registered trademark of Oracle and/or its affiliates in the US and other countries.
Copyright © 1993, 2026, Oracle and/or its affiliates, 500 Oracle Parkway, Redwood Shores, CA 94065 USA.
All rights reserved. Use is subject to license terms and the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.